

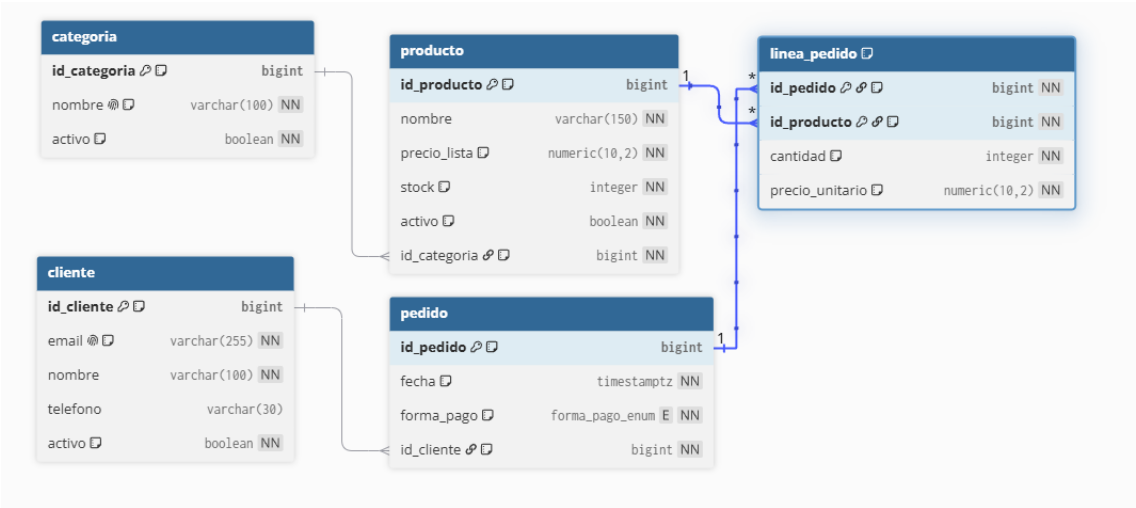
TRABAJO PRÁCTICO N.º 1

Del dominio al esquema: modelo entidad-relación, modelo relacional, normalización y DDL

Campo	Detalle
Materia	Base de Datos II
Carrera	Tecnicatura Universitaria en Programación a Distancia (UTN)
Estudiante	Deseff Facundo

Parte 1 — Modelo Entidad-Relación

1.1. Diagrama ER



1.2. Diccionario de Entidades y Atributos

ENTIDAD	ATRIBUTO	TIPO CONCEPTUAL	CLAVE / RESTRICCIÓN	DESCRIPCIÓN DEL DOMINIO
CATEGORIA	id_categoria	Numérico (Bigint)	PK Autogenerada	Identificador único de categoría.
	nombre	Texto (Varchar 100)	UNIQUE NOT NULL	Nombre de categoría (ej. Pizzas, Bebidas).
	activo	Booleano	DEFAULT TRUE NOT NULL	Baja lógica sin borrado físico (R7).
PRODUCTO	id_producto	Numérico (Bigint)	PK Autogenerada	Identificador único del producto.
	nombre	Texto (Varchar 150)	NOT NULL	Denominación comercial del producto.
	precio_lista	Numérico (10,2)	CHECK (precio >= 0)	Precio de venta actual de lista (R5).
	stock	Numérico (Integer)	CHECK (stock >= 0)	Stock disponible en inventario (R5).
	activo	Booleano	DEFAULT TRUE NOT NULL	Baja lógica sin borrado físico (R7).
	id_categoria	Numérico (Bigint)	FK NOT NULL	Categoría obligatoria a la que pertenece (R1).
CLIENTE	id_cliente	Numérico (Bigint)	PK Autogenerada	Identificador único del cliente.
	email	Texto (Varchar 255)	UNIQUE NOT NULL	Correo que identifica al cliente (R6).
	nombre	Texto (Varchar 100)	NOT NULL	Nombre completo del cliente.
	telefono	Texto (Varchar 30)	Opcional (NULL)	Teléfono de contacto del cliente.
	activo	Booleano	DEFAULT TRUE NOT NULL	Baja lógica de cliente.
PEDIDO	id_pedido	Numérico (Bigint)	PK Autogenerada	Identificador único del pedido.
	fecha	Timestamptz	DEFAULT now() NOT NULL	Fecha y hora exacta de emisión.
	forma_pago	Enumerado	NOT NULL	EFFECTIVO, TARJETA o TRANSFERENCIA.
	id_cliente	Numérico (Bigint)	FK NOT NULL	Cliente obligatorio que realizó el pedido (R2).
LINEA_PEDIDO (Relación N:M)	id_pedido	Numérico (Bigint)	PK Compuesta / FK	Pedido al que pertenece el ítem.
	id_producto	Numérico (Bigint)	PK Compuesta / FK	Producto vendido en la línea.
	cantidad	Numérico (Integer)	CHECK (cantidad > 0)	Unidades pedidas de la línea (R4).
	precio_unitario	Numérico (10,2)	CHECK (precio >= 0)	Precio histórico pactado al momento de venta (R4).

1.3. Respuestas a las Preguntas Guía (Parte 1)

- **¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?**

Si se hace 1:N de pedido a producto, un producto solo podría venderse en un único pedido en toda la historia y no se podría volver a vender. Si se hace 1:N de producto a pedido, un pedido solo podría tener un único producto. Además se perdería el registro de la cantidad vendida de cada ítem y el precio unitario histórico al momento de la venta.

- **¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?**

Categoría tiene participación parcial porque puede existir una categoría recién creada sin ningún producto cargado. Producto tiene participación total porque todo producto tiene que pertenecer obligatoriamente a una categoría. Si se invirtiera, quedarían productos sin categoría asignada y no se podría crear una categoría vacía sin meterle un producto al instante.

- **¿Alguno de tus atributos podría descomponerse en partes más simples (por ejemplo, un nombre completo en nombre y apellido)? Justificá tu decisión de dejarlo compuesto o de separarlo.**

El nombre del cliente se puede separar en nombre y apellido. Conviene separarlo para poder ordenar alfabéticamente por apellido y hacer búsquedas más cómodas en el sistema sin tener que manipular el texto desde la aplicación.

Parte 2 — Derivación al Modelo Relacional

2.1. Esquema Relacional Derivado

- **categoria** (id_categoria, nombre, activo)
 - PK: id_categoria
- **producto** (id_producto, nombre, precio_lista, stock, activo, id_categoria)
 - PK: id_producto
 - FK: id_categoria -> categoria(id_categoria)
- **cliente** (id_cliente, email, nombre, telefono, activo)
 - PK: id_cliente
- **pedido** (id_pedido, fecha, forma_pago, id_cliente)
 - PK: id_pedido
 - FK: id_cliente -> cliente(id_cliente)
- **linea_pedido** (id_pedido, id_producto, cantidad, precio_unitario)
 - PK compuesta: (id_pedido, id_producto)
 - FK 1: id_pedido -> pedido(id_pedido)
 - FK 2: id_producto -> producto(id_producto)

2.2. Justificación de la Clave Primaria en la Tabla Intermedia

Se eligió una clave primaria compuesta por (id_pedido, id_producto). Como por regla de negocio un producto no se repite en más de una línea dentro del mismo pedido, la combinación de ambos IDs identifica cada registro de forma única y evita duplicados sin necesidad de crear un id artificial extra.

2.3. Respuestas a las Preguntas Guía (Parte 2)

- **¿Qué pasaría con la integridad de tus datos si la tabla intermedia no incluyera ambas claves foráneas como NOT NULL?**

Se rompería la integridad referencial. Si permitieran nulos, podrían guardarse líneas de pedido huérfanas (un producto vendido sin saber de qué pedido es, o un pedido con un ítem que no tiene producto asignado), lo que arruinaría los reportes de ventas y el control de stock.

- **Si el pedido pudiera existir sin ningún producto asociado (por ejemplo, un pedido recién iniciado), ¿cambia esto la participación que definiste en la Parte 1 para la relación con la tabla intermedia? Explica el impacto.**

Sí, la participación de pedido pasaría a ser parcial al crearse. En la estructura de tablas no cambia nada, pero en el sistema permitiría hacer un INSERT en la tabla pedido antes de cargar las líneas en linea_pedido.

Parte 3 — Normalización hasta 3FN / BCNF

3.1. Relación universal y clave candidata

Conjunto de atributos de la planilla:

$R = \{\text{nro_pedido}, \text{fecha}, \text{cliente}, \text{producto}, \text{categoria}, \text{precio_unitario}, \text{cant}, \text{subtotal}, \text{forma_de_pago}\}$

- Clave candidata: {nro_pedido, producto}

3.2. Dependencias Funcionales

1. nro_pedido -> fecha, cliente, forma_de_pago
2. producto -> categoria
3. nro_pedido, producto -> cant, precio_unitario, subtotal

3.3. Paso a paso de normalización

- **Paso 1: Verificación de 1FN**

La tabla está en 1FN porque todos los campos tienen valores atómicos (no hay listas ni grupos repetidos en una misma celda).

- **Paso 2: Descomposición a 2FN**

Para estar en 2FN, ningún atributo no clave debe depender de una parte de la clave compuesta {nro_pedido, producto}.

Como {fecha, cliente, forma_de_pago} dependen solo de nro_pedido, y {categoria} depende solo de producto, se separa en tres tablas:

- PEDIDO_CLIENTE (nro_pedido, fecha, cliente, forma_de_pago)
- PRODUCTO_CAT (producto, categoria)
- LINEA_PEDIDO (nro_pedido, producto, cant, precio_unitario, subtotal)
- **Paso 3: Descomposición a 3FN**

Para estar en 3FN no debe haber dependencias transitivas entre atributos no clave.

En LINEA_PEDIDO, el subtotal se calcula multiplicando cant * precio_unitario (es un dato derivado). Se elimina la columna subtotal de la tabla física.

Las tablas quedan:

- PEDIDO (nro_pedido, fecha, cliente, forma_de_pago)

- PRODUCTO (producto, categoria)
- LINEA_PEDIDO (nro_pedido, producto, cant, precio_unitario)
- **Paso 4: Verificación de BCNF**

En todas las tablas resultantes, cada determinante es una clave candidata:

- En PEDIDO: nro_pedido es clave.
- En PRODUCTO: producto es clave.
- En LINEA_PEDIDO: (nro_pedido, producto) es clave.

Cumple con BCNF.

3.4. Preguntas de integración

- **Comparación entre normalizar la planilla y modelar desde el ER:**

Normalizar la planilla sirve para ordenar datos históricos que ya estaban cargados, pero le faltan datos importantes del negocio como el stock actual, email y teléfono del cliente, precio de lista base o la baja lógica de productos. Modelar con ER a partir del negocio es más completo porque contempla cómo funciona el sistema a futuro y no solo lo que quedó anotado en una planilla.

- **El atributo subtotal: ¿conviene almacenarlo o recalcularlo al vuelo?**

No conviene guardarlo como columna fija porque es un valor derivado ($\text{cant} * \text{precio_unitario}$) y rompería la 3FN generando datos redundantes. Conviene calcularlo directamente en las consultas SQL (`SELECT cant * precio_unitario AS subtotal`). Al tener guardado el precio_unitario de ese momento, el cálculo no cambia nunca.

Parte 4 — DDL en PostgreSQL (schema.sql)

-- Creacion del tipo enumerado para la forma de pago

```
CREATE TYPE forma_pago_enum AS ENUM ('EFFECTIVO', 'TARJETA', 'TRANSFERENCIA');
```

-- Tabla categoria

```
CREATE TABLE categoria (
```

```
    id_categoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    nombre VARCHAR(100) NOT NULL UNIQUE,
```

```
    activo BOOLEAN NOT NULL DEFAULT TRUE,
```

```
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);
```

-- Tabla cliente

```
CREATE TABLE cliente (  
    id_cliente BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    email VARCHAR(255) NOT NULL UNIQUE,  
    nombre VARCHAR(100) NOT NULL,  
    telefono VARCHAR(30),  
    activo BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

-- Tabla producto

```
CREATE TABLE producto (  
    id_producto BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nombre VARCHAR(150) NOT NULL,  
    precio_lista NUMERIC(10,2) NOT NULL,  
    stock INTEGER NOT NULL DEFAULT 0,  
    activo BOOLEAN NOT NULL DEFAULT TRUE,  
    id_categoria BIGINT NOT NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
    CONSTRAINT chk_producto_precio CHECK (precio_lista >= 0),  
    CONSTRAINT chk_producto_stock CHECK (stock >= 0),  
    CONSTRAINT fk_producto_categoria FOREIGN KEY (id_categoria)  
        REFERENCES categoria (id_categoria) ON DELETE RESTRICT  
);
```

-- Tabla pedido

```
CREATE TABLE pedido (  
    id_pedido BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    fecha TIMESTAMPTZ NOT NULL DEFAULT now(),  
    forma_pago forma_pago_enum NOT NULL,  
    id_cliente BIGINT NOT NULL,
```

```
CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente)
REFERENCES cliente (id_cliente) ON DELETE RESTRICT
);
```

-- Tabla intermedia linea_pedido

```
CREATE TABLE linea_pedido (
    id_pedido BIGINT NOT NULL,
    id_producto BIGINT NOT NULL,
    cantidad INTEGER NOT NULL,
    precio_unitario NUMERIC(10,2) NOT NULL,

    CONSTRAINT pk_linea_pedido PRIMARY KEY (id_pedido, id_producto),
    CONSTRAINT chk_linea_pedido_cantidad CHECK (cantidad > 0),
    CONSTRAINT chk_linea_pedido_precio CHECK (precio_unitario >= 0),
    CONSTRAINT fk_linea_pedido_pedido FOREIGN KEY (id_pedido)
        REFERENCES pedido (id_pedido) ON DELETE RESTRICT,
    CONSTRAINT fk_linea_pedido_producto FOREIGN KEY (id_producto)
        REFERENCES producto (id_producto) ON DELETE RESTRICT
);
```

-- Indices para acelerar consultas comunes

-- Acelera buscar los pedidos de un cliente

```
CREATE INDEX idx_pedido_cliente ON pedido (id_cliente);
```

-- Acelera listar productos activos por categoria

```
CREATE INDEX idx_producto_categoria_activo ON producto (id_categoria, activo);
```